

04 · Pi 的 Agent Loop：文本、工具调用与停止条件

拆解 runLoop 的双层循环、事件状态机与工具执行闭环

分析版本：027a5847901b5dde30270abaa1041046cd2b4b55

核心结论

Agent Loop 位于 packages/agent/src/agent-loop.ts。它是“双层循环”：内层处理当前工作（模型响应、工具调用、steering），外层在本来要停止时再检查 follow-up。普通文本且无工具调用会自然结束；toolCall 会执行后把 toolResult 追加上下文并继续；error/aborted、shouldStopAfterTurn、工具批次 terminate 等会改变结束路径。

1. Agent facade 与 loop engine 分工

Agent 类是有状态 facade：持有 transcript、model、tools、thinkingLevel、队列、activeRun/AbortController，并把 run 的输入转换为 AgentContext + AgentLoopConfig。真正的循环算法在 runAgentLoop/runLoop。这个拆分让低层算法可以脱离 UI/Session 单独测试。

2. runAgentLoop 的启动协议

1. 复制 prompts 到 newMessages。
2. 把 prompts 追加到 currentContext.messages。
3. 依次发出 agent_start、turn_start。
4. 对每条初始 prompt 发 message_start/message_end。
5. 进入 runLoop。

newMessages 与 currentContext.messages 不完全等价：前者是“本次 run 新产生的消息”，后者是“本轮 LLM 所见的当前上下文”，包含 run 之前的历史。agent_end 返回的是 newMessages。

3. 为什么是外层 + 内层两个 while

```
outer while(true):
  pending = steering?
  inner while(hasMoreToolCalls || pending.length):
    inject pending
    assistant = provider request
    execute tool calls
    prepare next turn / shouldStop
    pending = steering queue
  followUps = follow-up queue
  if followUps: pending = followUps; continue outer
  break
emit agent_end
```

steering 的语义是“当前工作尚未结束时改变接下来一轮”；follow-up 的语义是“Agent 本来准备停止后再追加任务”。因此它们必须在不同的 drain point 被轮询。

4. 普通文本响应路径

streamAssistantResponse 返回 AssistantMessage。若 content 中没有 toolCall，hasMoreToolCalls 保持 false。完成 turn_end 和 prepareNextTurn/shouldStopAfterTurn 后，只要没有 steering 和 follow-up，循环退出并发 agent_end。stopReason=stop/length 本身并不自动代表有下一轮；是否继续取决于工具/队列。

5. 工具调用路径

6. 从 assistant.content 过滤 type=toolCall。
7. 如果 stopReason=length，所有工具调用直接失败，不执行；原因是流式 JSON 可能被截断但“修复后看似合法”。
8. 否则 executeToolCalls，根据全局模式或任一工具 executionMode 选择 sequential/parallel。
9. 每个工具结果转换为 ToolResultMessage，追加 currentContext.messages 和 newMessages。
10. hasMoreToolCalls = !batch.terminate；通常继续请求模型，让模型消费 toolResult。

6. 工具批次 terminate 的语义

shouldTerminateToolBatch 只有在批次非空且“每一个 finalized result 都 terminate=true”时才返回 true。也就是说，一个工具想结束整个批次并不足够，必须所有结果都同意。这避免并行工具中某一个结果意外截断其他逻辑。

7. error / aborted 路径为什么仍然完整发事件

如果 assistant stopReason 是 error 或 aborted，runLoop 仍发 turn_end，再发 agent_end，然后返回。Agent 测试明确验证 provider 抛异常时也被规范化成 assistant error message，并产生完整生命周期事件。这使 UI/Session 不需要用 try/catch 猜测 run 是否异常结束。

8. provider stream 事件如何变成 AgentEvent

streamAssistantResponse 消费 pi-ai AssistantMessageEventStream：start -> message_start；text/thinking/toolcall delta -> message_update；done/error -> result() 得到最终 AssistantMessage 并发 message_end。partial message 始终被放在 context.messages 的最后一位并原位替换。

9. 停止条件总表

条件	发生位置	结果
assistant error/aborted	每次 provider 响应后	turn_end + agent_end，立即结束

无 toolCall、无 steering、无 follow-up	双层循环自然耗尽	agent_end
tool batch terminate=true	执行工具后	不再因 toolCall 自动继续；仍检查后续逻辑/队列
shouldStopAfterTurn 返回 true	turn_end 后	跳过 steering/follow-up，直接 agent_end
steering queue 非空	每个 turn 后	注入为新消息，再请求模型
follow-up queue 非空	Agent 原本停止点	重新进入 outer loop
AbortSignal	provider/tool/hook	最终变成 aborted/error 流或工具错误

10. 状态不变量

- LLM provider 调用前，Context.messages 最后一条必须是 provider 可接受的 user/toolResult 等角色；continue() 对 assistant 尾消息有保护。
- toolResult 必须与此前 toolCall 关联；loop 把结果在下一次 provider 请求前追加上下文。
- Agent 同一时刻只允许一个 activeRun；新输入应走 steer/followUp 或等待。
- 异步 event subscriber 是 run settlement 的一部分；prompt() 不会在 agent_end listener 尚未完成时提前 resolve。
- late tool update 在工具 settle 后被忽略，避免已结束工具继续污染事件流。

11. 性能与并发权衡

默认 parallel 工具模式先顺序做 preflight，再并发执行允许的工具；tool_execution_end 可以按实际完成顺序出现，但 ToolResultMessage 最后仍按 assistant 原始 tool-call 顺序发出。这样兼顾延迟与上下文确定性。若任一工具标记 executionMode=sequential，则整批切换到顺序模式，适合存在文件写入互斥或依赖关系的工具。

动态验证方案

实验 1：普通文本一次结束

假设：无工具调用时只发生一次 provider 请求

操作：注入 Mock stream 返回 stop 文本，订阅 AgentEvent。

预期：agent_start -> turn_start -> user start/end -> assistant start/end -> turn_end -> agent_end；provider 调用 1 次。

若不一致：若多一次调用，检查 steering/follow-up 或 prepareNextTurn。

实验 2：工具闭环

假设：toolUse 会执行工具并再次请求模型

操作：第一次 mock 返回 toolCall，工具返回文本；第二次 mock 返回 stop。

预期：看到 tool_execution_*、toolResult message，然后第二个 turn。

若不一致：若工具未执行，检查 tool name、schema validation、beforeToolCall block。

实验 3：并行顺序不变量

假设：parallel 允许不同完成顺序，但 result artifact 按 source order

操作：两个工具设置不同延迟，记录 end events 与 toolResult message 顺序。

预期：end 可乱序，toolResult message 保持模型调用顺序。

若不一致：若结果顺序改变，可能影响 Provider 对 tool-call pairing 的预期。

源码证据索引

文件	关键符号/用途	本报告结论
packages/agent/src/agent.ts	Agent / activeRun / queues / runPromptMessages	状态 facade 与生命周期
packages/agent/src/agent-loop.ts	runAgentLoop / runLoop / streamAssistantResponse	核心双层循环
packages/agent/src/agent-loop.ts	executeToolCalls / prepareToolCall / finalizeExecutedToolCall	工具闭环
packages/agent/src/types.ts	AgentLoopConfig / StreamFn / ToolExecutionMode	loop 的依赖注入契约
packages/agent/test/agent.test.ts	lifecycle / abort / async subscriber / late update	实际行为契约

证据状态：除非单独标注，本报告中的行为结论均来自上述固定 commit 的静态源码与仓库测试/文档交叉验证；未实际启动 Pi、未抓取真实网络请求，因此“运行结果已验证”项为空。

推荐阅读顺序

优先级	文件	阅读目的	精读
1	packages/agent/src/agent.ts	先看状态/入口，不要一开始钻循环	是
2	packages/agent/src/agent-loop.ts	精读 runLoop	是
3	packages/agent/src/types.ts	理解 hook/队列/StreamFn 契约	是
4	packages/agent/test/agent.test.ts	用测试校验事件和并发理解	是